

Requirements Description

Node.js Performance Analyser Project



DSD Course



1. INTRODUCTION.....	3
1.1 Purpose.....	3
1.1.1 Goals.....	3
1.2 Scope.....	3
1.3 Revision history.....	4
2. Requirements.....	5
2.1 Requirements Definition.....	5
2.1.1 Functional Requirements.....	5
2.1.2 Non-Functional Requirements.....	9
2.2 User Stories.....	10
2.3 Mapping of requirements to user stories.....	21

1. INTRODUCTION

1.1 Purpose

The Node.js runtime is one of the most used technologies for developing applications, allowing users to execute code written in the very popular JavaScript programming language on the server side. However, when a Node.js application starts showing performance issues, developers often struggle to visualize what metrics are causing the most impact, and what configurations of the runtime should be changed to optimize the application. The purpose of the project is to remedy those struggles by developing a web application to assist the developers by allowing them to simulate the analysis of a selection of applications via visualization of the most important metrics, and evaluating the impact of each configuration.

1.1.1 Goals

- [G1] Developers, Site Reliability Engineers (SREs), and performance engineers want to easily explore and analyze performance data of Node.js applications through an intuitive web interface.
- [G2] Developers can choose from a selection of applications and visualize their most important metrics and configurations through interactive dashboards and graphs.
- [G3] Developers shall be informed about risks and problems detected on the selected application, such as anomalous patterns, bottlenecks, memory leaks, suboptimal configurations and other indicators.
- [G4] Users would like to receive tailored optimization recommendations based on identified issues, including suggestions for parameter tuning, heap size adjustments, and other configuration changes that improve performance and stability.

1.2 Scope

The Node.js Performance Analyzer is a web-based platform designed to assist developers, Site Reliability Engineers (SREs), and performance engineers in evaluating and optimizing the performance of Node.js applications. The system provides an interactive environment where users can explore pre-collected performance data, identify performance issues, and receive tailored optimization recommendations.

Users can begin by selecting an application and its configuration from a dataset containing multiple Node.js applications executed under various heap and memory settings. Once an application is selected, the platform analyzes its performance metrics—such as CPU utilization, memory consumption, and garbage collection activity—and presents them through interactive

dashboards. These visualizations enable users to quickly identify trends, anomalies, or inefficiencies in the application's runtime behavior.

Beyond visualization, the system automatically detects potential performance problems, including bottlenecks, memory leaks, and suboptimal configurations. Based on these findings, it generates actionable recommendations such as adjusting heap size, tuning garbage collection parameters, or modifying thread pool configurations to enhance efficiency and stability.

The platform is composed of two main components: a frontend dashboard for visualization and interaction, and a backend analysis engine that processes performance data and applies analytical or rule-based methods to detect issues. The dataset and analysis are fully simulated; the system does not connect to live applications but instead relies on existing datasets to replicate realistic performance analysis scenarios.

The Node.js Performance Analyzer aims to simplify the process of performance optimization by bridging the gap between raw runtime data and practical tuning decisions. It provides a learning and experimentation environment for understanding Node.js performance dynamics and can be extended in the future with advanced analytics, authentication, or enterprise features.

1.3 Revision history

Version	Date	Description	Author
1.0	31-10-2025	Initial version of the RASD document	<i>Nodejs Performance Analyzer Team</i>

2. Requirements

2.1 Requirements Definition

2.1.1 Functional Requirements

ID	Name	Description
	Data Selection	
FR-01	Web Interface Access	The system must allow the user to initiate the performance analysis of Node.js applications through a functional web interface.
FR-02	Available Application Listing	The system must allow the user to view a complete list of all available applications sourced from the provided dataset.
FR-03	Application Selection	The system must allow the user to select one application from the displayed list.
FR-04	Execution Parameter Display	The system must allow the user to view all available execution parameters relevant to the selected application (e.g., heap size, thread pool size).
FR-05	Parameter Selection Capability	The system must allow the user to select one or more execution parameters to include in the performance analysis run.
FR-06	Source Data Loading	The system must read and load the raw timeseries data for the selected application run directly from a data source that contains the complete dataset.
	Data Processing (Backend Processing)	
FR-07	Core Metric Calculation	The system must execute processes to calculate core performance metrics for the selected run, specifically CPU usage and total memory utilization.
FR-08	Detailed Memory Processing	The system must process the loaded data to calculate and structure the detailed memory components of the Node.js runtime, including the internal heap space breakdown.

FR-09	GC Statistics Processing	The system must process the source data to derive and calculate comprehensive Garbage Collection (GC) statistics (collection count, pause time, scavenge vs mark-sweep cycles).
FR-10	Data Preparation for Context	The system must process the calculated timeseries data to identify and timestamp significant events (such as GC events) in a structured format, ready for contextual visualization.
FR-11	Structuring for Interactivity (API)	The Backend must aggregate and structure the calculated performance metrics and expose them via an API that supports interactive exploration and configuration comparison.
	Visualization & Interaction (Frontend)	
FR-12	Dashboard Visualization	The system must present all analyzed metrics via interactive dashboards utilizing appropriate chart types, including line charts, scatter plots, heatmaps, and stacked area charts.
FR-13	Core Metrics Display	The system allows the user to view time-series charts presenting the core performance metrics, specifically CPU usage and the primary memory consumption metrics (heap used, heap total, RSS).
FR-14	Heap Breakdown Visualization	The system allows the user to view the detailed heap space breakdown presented as a dedicated time-series visualization.
FR-15	GC Activity Context Display	The system allows the user to view memory usage time-series data augmented with visual markers or indicators for Garbage Collection events.
FR-16	GC Statistics Display	The system allows the user to view the calculated detailed Garbage Collection (GC) statistics through graphical and numerical displays.
FR-17	Usability Context Display	The system allows the user to access clear visual context on all charts, provided through axis labels, legends, and tooltips with metric explanations.
FR-18	Time Window Manipulation	The system allows the user to manipulate the displayed timeseries data.

FR-19	Time Window Exploration	The system allows the user to explore arbitrary time windows within the collected dataset.
FR-20	Zoom Capability	The system allows the user to zoom into specific periods of interest in the timeseries data.
FR-21	Correlation Selection	The system allows the user to select pairs of available performance metrics for correlation analysis (e.g., CPU usage and heap usage).
FR-22	Correlation Visualization	The system allows the user to view and visualize correlations between the selected metric pairs using appropriate chart types.
FR-23	Configuration Comparison	The system allows the user to select and compare performance metrics between different configurations of the same application (e.g., comparing heap size 1024MB vs 2048MB) using the visualization tools.
I	Analysis & Recommendations (Analysis Engine)	
FR-24	Absolute Value Anomaly Detection	The system must analyze the absolute metric values collected and flag predefined indicators of poor performance, resource limitations, or suboptimal configurations.
FR-25	Memory Leak Trend Analysis	The system must execute trend analysis on memory consumption metrics (such as heap usage and RSS) over defined periods to detect and flag patterns consistent with potential memory leaks.
FR-26	Correlated Issue Analysis	The system must analyze the correlations between collected metrics to detect related performance issues or bottlenecks
IFR-27	Optimization Recommendation Generation	The system must generate specific, actionable recommendations for configuration changes (e.g., increase heap size, GC parameter adjustments) based on the comprehensive analysis of all identified problems.
FR-28	Plain-Language Explanation	The system must ensure that every generated optimization recommendation is presented in plain language with an explanation that makes the insight actionable for developers and SREs.
	User Workflow	

FR-29	Guided Analysis Workflow	The system must guide the user sequentially through the defined performance analysis steps (Select application → Select configuration → Explore metrics → Identification of issues → Presentation of recommendations).
-------	--------------------------	--

2.1.2 Non-Functional Requirements

ID	Name	Description
NFR-01	Intuitive Interface	The user interface must be designed to be intuitive and easily navigated by developers who do not possess deep, specialized knowledge of the Node.js runtime environment.
NFR-02	Contextual Visualization Support	All visualizations must provide essential context including clear axis labels, comprehensive legends, and hover tooltips containing explanations for the displayed metrics and data points.
NFR-03	Efficient Large Dataset Rendering	The system must efficiently handle large time-series datasets, ensuring that visualizations load quickly and remain responsive. Initial load time should be under three seconds for up to 100,000 data points, while larger datasets should use techniques like lazy loading or pagination.
NFR-04	Architectural Extensibility	The system's underlying architecture must be structured to enable the future integration of additional performance metrics or new analysis modules, ensuring that the detection logic is pluggable to accommodate new rules or algorithms without requiring major refactoring efforts.
NFR-05	Standardized Deployment	The system must support containerized deployment via Docker, and the deliverable must include clear, up-to-date documentation detailing all dependencies and setup procedures.
NFR-06	Basic Interface Security	The system must enforce basic security measures for the web interface and data access points, including the use of secure protocols (e.g., HTTPS) for all API requests to protect data transmission.

2.2 User Stories

ID: NJS-1

Title: Web Interface for Performance Analysis

Upon launching the system:

- The system displays a login or access screen (if authentication is required).
- After successful access, the system presents the main dashboard within a web browser.
- The user selects the option to start a new performance analysis or view existing analyses.
- If starting a new analysis, the system prompts the user to choose an application from the available dataset.
- Once an application is selected, the system allows the user to configure analysis parameters (e.g., heap size, thread pool size).
- The user initiates the performance analysis run.
- The system begins processing and displays progress indicators in real time.
- Upon completion, the system updates the dashboard to show summarized results and access to detailed metric visualizations (CPU, memory, GC statistics).
- The user can continue to monitor or revisit these analyses from the same web interface at any time.

ID: NJS-2

Title: Application Listing and Selection

Upon accessing the analysis dashboard:

- The system loads and displays a list of all available Node.js applications sourced from the dataset.
- The user reviews the displayed list to identify the desired application.
- The system provides search, filter, or sort options to assist in finding a specific application.
- The user selects one application for analysis.
- The system confirms the selection and retrieves configuration details associated with the chosen application.
- The system updates the interface to display execution parameters related to that application.

ID: NJS-3

Title: Application Listing and Selection (Alternate User Role Perspective)

Upon preparing to perform a performance analysis:

- The system presents a comprehensive list of available Node.js applications derived from the dataset.
- The user reviews the list and identifies the target application.
- The user selects the desired application and its associated configuration.
- The system validates the selection and loads configuration metadata required for analysis.
- The system transitions to the next step, displaying all relevant execution parameters for the selected application.

ID: NJS-4

Title: View Execution Parameters

Upon the user selecting an application for analysis:

- The system retrieves all execution parameters related to the chosen application, including heap size, thread pool size, and RAM allocation.
- The system displays these parameters in a structured and readable format within the interface.
- The user reviews the parameters to understand the current configuration context.
- The system allows the user to modify or confirm the parameters for the upcoming analysis run.
- Upon confirmation, the system stores the chosen parameters and prepares for metric processing.

ID: NJS-5

Title: Processing and Visualizing Performance Metrics

Upon the user initiating a performance analysis:

- The system begins processing the selected application's performance data.
- The system aggregates key metrics, including CPU usage, memory consumption (heap used, heap total, RSS), and garbage collection statistics.
- The system visualizes these metrics in interactive time-series charts.
- The user observes CPU and memory usage patterns over time to identify performance trends.
- The system provides additional chart controls for zooming and exploring specific time windows.
- Upon completion, the system stores the processed metrics and allows the user to access or export the results for further analysis.

ID: NJS-6**Title:** Explore Time Ranges and Correlations

Upon viewing performance metrics:

- The system displays all time-series visualizations for CPU, memory, and garbage collection activity.
- The user selects a specific time range of interest within any chart.
- The system dynamically zooms into the selected time window and reloads detailed metric data for that interval.
- The user chooses one or more additional metrics (e.g., heap size, GC pause time) to compare.
- The system generates correlation plots (e.g., CPU vs. memory usage) to help the user identify relationships between metrics.
- The user reviews the correlation charts and time-specific data trends to understand how various runtime factors affect performance.

ID: NJS-7**Title:** Performance Issue Detection

Upon completion of metric processing:

- The system analyzes collected metrics for anomalies such as irregular CPU spikes, memory leaks, or sustained garbage collection pauses.
- The system flags unusual patterns and visually marks them on the time-series charts.
- The user views highlighted sections indicating performance anomalies.
- The system provides summaries describing detected issues (e.g., "Possible memory leak detected").
- The user reviews these insights to focus further analysis on problematic regions or configurations.

ID: NJS-8**Title:** Optimization Recommendations

Upon detection of performance issues:

- The system evaluates detected anomalies to determine relevant optimization opportunities.
- The system generates recommendations covering Node.js runtime tuning (e.g., GC parameters, thread pool configuration).
- Each recommendation is presented in plain, human-readable language.
- The user reviews the recommendations and optionally applies them to future test runs or configurations.

ID: NJS-9

Title: Guided Workflow

Upon accessing the analysis dashboard:

- The system presents a guided, step-by-step workflow to help the user perform a complete performance analysis.
- Step 1: Select an application from the dataset.
- Step 2: Choose or adjust configuration parameters for the analysis.
- Step 3: Execute and explore performance metrics via interactive charts.
- Step 4: Identify detected issues flagged by the system.
- Step 5: Review automatically generated recommendations.
- The system highlights the current step and disables unavailable actions until prerequisites are completed.

ID: NJS-10

Title: Work Efficiently with Large Datasets

Upon loading large time-series datasets:

- The system implements lazy loading or pagination to fetch data in segments.
- The system displays placeholder indicators while new data is being retrieved.
- The dashboard remains responsive during data loading and chart rendering.
- The user can still interact with loaded charts while additional data continues to load in the background.
- The system ensures smooth panning, zooming, and filtering operations without freezing or long delays.

ID: NJS-11**Title:** Receive Practical Tuning Suggestions

Upon completion of issue detection:

- The system evaluates the identified performance bottlenecks.
- The system generates targeted configuration suggestions (e.g., adjust GC interval, increase heap limit).
- The system presents the suggestions in plain language along with technical explanations.
- The user reviews and applies selected tuning actions.
- The system tracks applied changes for future performance comparison.

ID: NJS-12**Title:** Refresh Dataset View

Upon selecting the refresh option:

- The system re-queries the dataset repository for available Node.js applications.
- Newly added applications become visible in the application list.
- The user confirms the presence of new items.
- The system maintains prior filters or sorting preferences unless reset.

ID: NJS-13**Title:** Reset Selections

Upon selecting the “Reset” command:

- The system clears all current application, configuration, and parameter selections.
- The dataset view returns to its default state.
- The user can reinitiate a new analysis process from a clean interface.
- If newly added applications exist, they remain visible after reset.

ID: NJS-14**Title:** Filtered Metric Loading

Upon selecting desired metrics:

- The system limits data retrieval to the chosen metrics (e.g., CPU, heap, GC).
- The system skips non-selected metrics to reduce processing time.
- The user observes faster loading and visualization performance.
- The system allows adjustments to the selection and reloads metrics accordingly.

ID: NJS-15**Title:** Change Chart Type

Upon viewing metric visualizations:

- The system displays the current chart format (e.g., line, scatter, stacked area, or heatmap).
- The user selects a control option to change the chart type.
- The system immediately re-renders the data using the newly selected visualization type.
- The system preserves the same dataset, time window, and filters while switching formats.
- The user evaluates the visualization to determine which chart type best highlights performance behavior.

ID: NJS-16**Title:** Download Snapshot

Upon reviewing a dashboard visualization:

- The user selects the option to export or download the current dashboard view.
- The system captures a high-resolution image (or PDF) of the active charts and metrics.
- The system prompts the user to confirm the filename and download location.
- The snapshot is downloaded locally and includes labels, legends, and timestamps for traceability.
- The user can then attach or share the exported visualization in reports or documentation.

ID: NJS-17**Title:** Multi-Application View

Upon selecting the comparison feature:

- The system displays a list of available Node.js applications in the dataset.
- The user selects two or more applications for side-by-side analysis.
- The system loads and aligns performance metrics from all chosen applications on a unified dashboard.
- The charts render synchronized time axes to facilitate direct visual comparison.
- The user reviews differences in performance metrics across applications to identify relative efficiency or bottlenecks.

ID: NJS-18**Title:** Show Expected Improvements

Upon generating optimization recommendations:

- The system calculates the estimated performance gains associated with each recommendation (e.g., reduced memory usage, lower GC pause times).
- The system displays these projected improvements beside each recommendation in percentage or numerical form.
- The user reviews the expected benefit to prioritize which optimizations to apply first.
- The system updates the recommendations list dynamically if parameter values change.

ID: NJS-19**Title:** Compare Before and After

Upon applying configuration fixes or optimizations:

- The system retrieves stored metric data from previous analysis runs.
- The user selects two runs—one before optimization and one after.
- The system overlays or aligns the pre- and post-fix metrics (e.g., memory usage, GC frequency) on comparative charts.
- The user reviews visual and numerical differences to confirm improvements.
- The system highlights the percentage changes in performance metrics for clarity.

Non-Functional Stories

ID: NJS-20

Title: Clear and contextual visualization

Description: As a user analyzing performance metrics, I want each chart to include clear axis labels, legends, and tooltips so that I can easily understand what each metric represents without confusion.

ID: NJS-21

Title: Simple and intuitive interface

Description: As a developer, I want the interface to be simple and easy to navigate, so that I can perform performance analysis tasks without needing additional training or documentation.

ID: NJS-22

Title: Containerized deployment

Description: As a developer or system administrator, I want to deploy the application easily using Docker with clear setup documentation, so that I can install and run the system quickly without complex configuration.

ID: NJS-23

Title: Extensible architecture

Description: As a developer, I want the system's architecture to support adding new metrics or analysis modules easily, so that future improvements can be made without major code changes.

ID: NJS-24

Title: Secure data access

Description: As a user, I want all communication between my browser and the system to be secured via HTTPS, so that my data and analysis results remain protected.

ID: NJS-25

Title: Responsive Visualization of Large Datasets

Description: As a user analyzing performance metrics, I want the system to load and render large time-series visualizations efficiently, so that charts appear quickly and interactive actions like zooming and filtering remain smooth and responsive.

2.3 Mapping of requirements to user stories

User Story ID	Mapped Requirement ID(s)
NJS-1	FR-01, FR-29
NJS-2	FR-02, FR-03
NJS-3	FR-02, FR-03, FR-05
NJS-4	FR-04, FR-05
NJS-5	FR-06, FR-07, FR-08, FR-09, FR-12, FR-13, FR-15
NJS-6	FR-18, FR-19, FR-20, FR-21, FR-22
NJS-7	FR-24, FR-25, FR-26
NJS-8	FR-27, FR-28
NJS-9	FR-29
NJS-10	NFR-03
NJS-11	FR-27, FR-28
NJS-12	FR-02
NJS-13	FR-02, FR-03
NJS-14	FR-06, FR-12, FR-13
NJS-15	FR-12, FR-13, FR-22
NJS-16	FR-12, FR-17
NJS-17	FR-23
NJS-18	FR-27, FR-28
NJS-19	FR-23
NJS-20	NFR-02
NJS-21	NFR-01
NJS-22	NFR-05

NJS-23	NFR-04
NJS-24	NFR-06
NJS-25	NFR-03